Rapport de projet :

Extraction de graphes de connaissances à partir de texte

Réalisé par :

- MECHERAK Thanina
- MOUHOUN LYDIA
- FELLATI ZINEB
- TAHIRI MOUAD

Encadré par:
Pr. KEDAD Zoubida

UVSQ M2 Datascale

Table de matières

Table de matières 2
I. Description générale des méthodes étudiées 4
1.1) Knowledge graphs can be constructed from unstructured texts to support learning environments (Agrawal et al., 2023) 4
1.2) Structured information can be extracted from free text corpora and queried using KGQA techniques (Atzori, 2022) 6
Contribution principale 7
1.3) Automatic ontology extraction methods can be used to transform textual documents into structured knowledge (Novelli & Oliveira, 2011) 8
1.4) Scalable techniques have been proposed to construct graphs from large text corpora (Debatty et al., 2014) 10
1. Les nœuds (les sommets) 11
2. Les arêtes (les liens) 11
3. Les poids (la force du lien) 11
À quoi ça sert une fois qu'on a ce graphe ? 11
1. La métrique de similarité textuelle 12
2. Le mécanisme de calcul (le processus) 12
3. La sélection des voisins (k-NN) 12
2) Etude comparative des méthodes : 13
II. Descriptif détaillé de la solution proposée pour la génération d'un graphe RDF à partir d'un texte: 17
1. Objectif général et principes directeurs 17
2. Vue d'ensemble du pipeline 17
3. Entrée utilisateur et configuration 18
3.1 Texte d'entrée 18
3.2 Paramétrage utilisateur 18
4. Prétraitement linguistique 19
4.1 Objectif 19
4.2 Méthodes utilisées 19
5. Extraction des entités nommées (NER) 19
5.1 Rôle dans le graphe de connaissances 19
5.2 Approche retenue 19
5.3 Résultat intermédiaire 20
6. Extraction des relations (composant central) 20
6.1 Niveau 1 – Extraction syntaxique à base de règles 20
6.2 Niveau 2 – Consolidation sémantique par LLM 21
7. Augmentation sémantique par LLM 22
7.1 Objectif 22

7.2 Augmentation des relations....22
7.3 Augmentation des entités : inférence de types conceptuels....23
8. Alignement ontologique.... 24
8.1 Mode Découverte (sans ontologie fournie)....24
8.2 Mode Alignement (ontologie fournie).... 24
9. Validation humaine (Human-in-the-loop).... 24
10. Génération, stockage et export du graphe....25
10.1 Génération RDF.... 25
10.2 Stockage.... 25
10.3 Formats d'export.... 25
11. Implémentation.... 25
Limites et perspectives.... 31

I. Description générale des méthodes étudiées

1.1) Knowledge graphs can be constructed from unstructured texts to support learning environments (Agrawal et al., 2023).

La construction de graphes de connaissances est difficile car elle vise à transformer un texte humain non structuré en une structure de données formelle et rigide.

- Les approches entièrement automatiques montrent leurs limites : les algorithmes de traitement du langage se basent sur la grammaire mais ne comprennent pas réellement le contexte du domaine.

- L'ambiguïté des termes complique l'extraction des connaissances, car un même mot peut avoir plusieurs sens selon son usage.

- La difficulté est renforcée par le volume important et la diversité des sources d'information (cours, laboratoires, documents techniques).

- Le problème principal réside dans l'absence d'une ontologie standard et de jeux de données étiquetés pour l'enseignement de la cybersécurité, pourtant indispensables pour construire un graphe cohérent et fiable.

La méthode proposée vise à construire des graphes de connaissances à partir de textes non structurés dans le domaine de la cybersécurité. Elle repose sur une approche *bottom-up* qui combine des techniques automatiques de traitement du langage naturel et l'intervention d'experts humains. Cette combinaison permet de structurer progressivement l'information, malgré l'absence de l'ontologie et de jeux de données préexistants.

Phase 1 : Acquisition et intégration des connaissances

Cette étape transforme les textes bruts (manuels de labo, notes) en données exploitables :

- • Extraction automatique (NLP) : Utilisation du traitement du langage naturel pour segmenter les phrases et identifier les entités (sujets/objets) et les relations (verbes/racines) via le marquage syntaxique (POS tagging).
- • Graphes préliminaires : Ces extractions brutes créent des triplets "entité-relation-entité" qui servent de base visuelle pour l'expert.(Phishing) — utilise → (Email frauduleux)
- • Développement de l'ontologie : Un expert humain affine ces données pour définir des catégories (concept, application, rôles, cours) et des règles strictes afin d'éliminer les ambiguïtés. Par exemple phishing est défini comme une attaque, l'email comme un outil utilisé pour transmettre le message frauduleux, et les identifiants comme des données sensibles ciblées par l'attaque.

- • Entity Matcher : UUn script Python personnalisé utilisant la bibliothèque spaCy est utilisé pour repérer automatiquement les concepts clés dans de nouveaux documents sans avoir à tout

étiqueter manuellement, par exemple dans la phrase « le spear phishing cible des employés via des emails personnalisés », le système identifie spear phishing comme une attaque, employés comme une cible et emails personnalisés comme une méthode utilisée pour réaliser l'attaque.

Phase 2 : Stockage des connaissances

Une fois les données structurées, elles doivent être conservées de manière efficace :

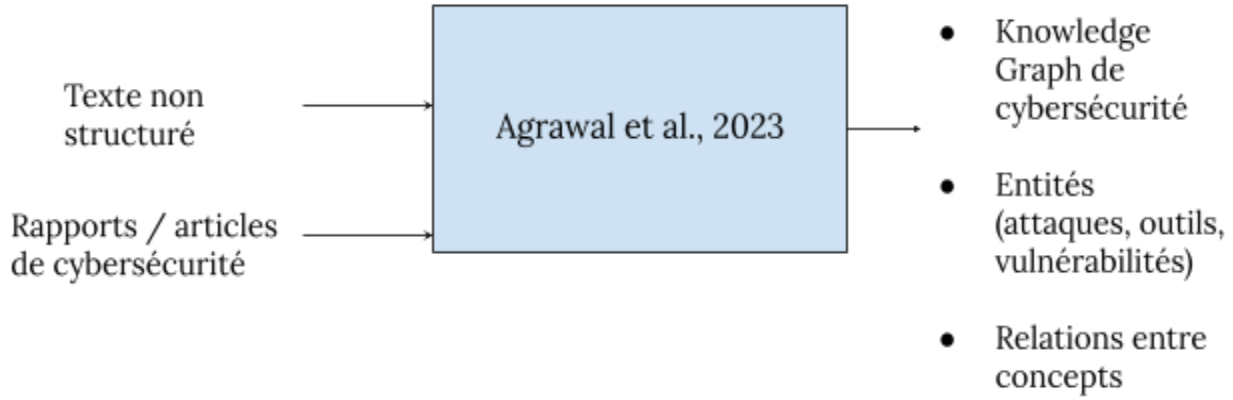
- • Base de données orientée graphe : Les sources utilisent Neo4j pour stocker les triplets, car ce système est flexible et permet d'ajouter facilement de nouveaux attributs aux nœuds et aux arcs.
- • Validation du schéma : La bibliothèque ReGraph est utilisée pour s'assurer que chaque nouvelle donnée insérée respecte l'ontologie définie par les experts (cohérence des types de relations, par exemple).

Phase 3 : Couche de consommation

C'est la partie visible pour l'utilisateur final (l'étudiant) :

- • Visualisation (Web UI) : Une interface web permet aux étudiants de consulter des sous-graphes spécifiques à leurs tâches pour mieux comprendre les flux de travail complexes.
- • Chatbot interactif : Un système de questions-réponses basé sur un modèle de machine learning (SVM - Support Vector Machine) est entraîné pour classer les intentions des étudiants et fournir des réponses précises extraites du graphe.

Cette méthode permet de passer d'un texte désordonné à un outil pédagogique structuré tout en limitant l'effort manuel massif grâce à l'automatisation partielle.



1.2) Structured information can be extracted from free text corpora and queried using KGQA techniques (Atzori, 2022).

Les auteurs présentent OKgraph, une librairie Python permettant d'extraire automatiquement un Knowledge Graph à partir de textes bruts.

Caractéristiques clés :

- approche totalement non supervisée
- aucune ontologie préalable requise
- utilisation d'embeddings de mots pour capter les relations sémantiques

OKgraph permet notamment :

- d'identifier des entités similaires (ex. villes, pays)
- de donner un label à ces groupes (ex. pays, capitales)
- d'extraire des relations entre entités
- de nommer ces relations

Le résultat est un Knowledge Graph léger, induit directement depuis le texte.

Interrogation des Knowledge Graphs

Extraire un graphe ne suffit pas. Le paper propose deux solutions pour l'exploiter facilement :

1. KGQA (Knowledge Graph Question Answering)
   L'utilisateur pose une question en langage naturel.
   Le système la traduit automatiquement en requête SPARQL.

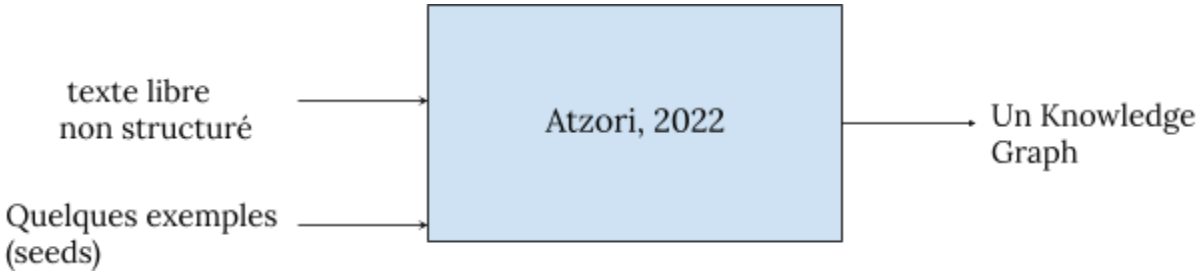
2. BESt Queries (By Example Structured Queries)
   L'utilisateur remplit ou modifie des champs structurés (type infobox Wikipédia).
   Ces champs sont transformés automatiquement en requêtes sur le graphe.

Ces deux approches permettent à des utilisateurs non techniques d'accéder à des données structurées complexes.

Contribution principale

La contribution du papier est double :

- montrer qu'il est possible de construire automatiquement un Knowledge Graph à partir de texte libre
- proposer des interfaces d'interrogation simples, accessibles sans expertise technique



Limitations du travail

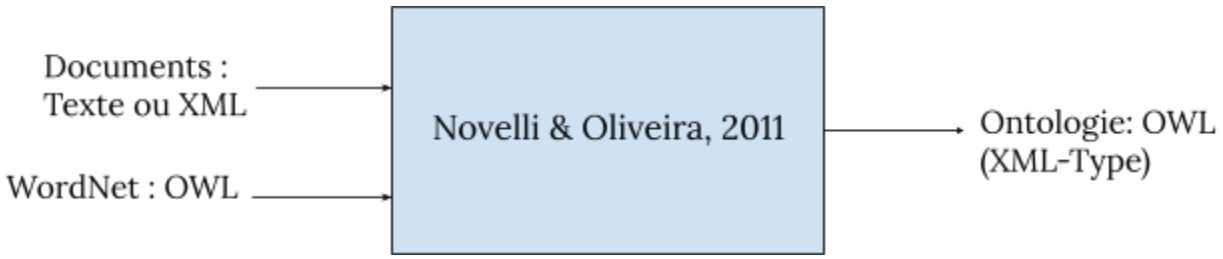
- La qualité du Knowledge Graph dépend fortement de la qualité des embeddings et du corpus utilisé.
- L'ontologie produite est peu formelle, sans contraintes logiques fortes.
- La gestion de l'ambiguïté sémantique est limitée à la similarité vectorielle.
- Il n'existe pas de validation formelle de la cohérence globale du graphe.

1.3) Automatic ontology extraction methods can be used to transform textual documents into structured knowledge (Novelli & Oliveira, 2011).

Contexte : Avec l'augmentation continue de la volumétrie et de la diversité des documents disponibles, notamment sur le web ou dans des bases de données, il devient essentiel de représenter et d'analyser les concepts qu'ils contiennent. En particulier, dans le domaine de la recherche d'informations, la sémantique joue un rôle fondamental pour améliorer la précision et la pertinence des systèmes de récupération documentaire.

Problématique : La croissance exponentielle du nombre et de la variété des documents rend l'analyse sémantique via les ontologies de plus en plus complexes et coûteuses, notamment lorsqu'il s'agit de générer ces ontologies de façon automatique, sans intervention humaine ou expertise approfondie. Les techniques avancées d'Intelligence Artificielle (IA), langage et contexte sont souvent coûteuses ou nécessitent une expertise spécialisée.

Contributions et démarche proposée : L'article présente une méthode novatrice mais simple, automatisée et agnostique quant au type précis de document (textes bruts ou XML). La démarche repose sur la combinaison efficace de plusieurs techniques, sans recours à une analyse approfondie du langage ou du contexte via des techniques sophistiquées d'IA.



Elle s'articule en plusieurs étapes :

1. **Pré-traitement des documents :**
- Pour les documents longs, il est créé un résumé afin de réduire leur taille.
- Pour les documents au format XML, une séparation entre structure et contenu est effectuée pour ne conserver que les éléments pertinents pour l'ontologie.
2. **Extraction de termes :**
- La constitution d'un ensemble de termes « index » est réalisée à partir de l'analyse TF-IDF, en conservant ceux présents dans plus de 25 % des documents.
3. **Analyse sémantique :**
- Utilisation de l'analyse sémantique latente (Latent Semantic Analysis, LSA) appliquée à une matrice terme-document pour capter la structure sémantique sous-jacente.
- La décomposition par SVD (Singular Value Decomposition) permet une réduction dimensionnelle et la création automatique de concepts.

- Chaque concept est défini par une union de jusqu'à trois termes issus des colonnes de la matrice U, qui représentent des groupes sémantiques cohérents.

4. **Construction de la hiérarchie :**

- Utilisation d'algorithmes de clustering hiérarchique (K-Means ou algorithme agglomératif) pour organiser les concepts en plusieurs niveaux, allant des termes de base à des concepts plus abstraits.

5. **Relation sémantique et liens :**

- Application de WordNet pour établir des relations sémantiques (synonymie, antonymie, hiérarchie) entre les concepts.
- Calcul de similarité sémantique, avec un seuil de 0,90, pour identifier des relations d'équivalence ou de hiérarchie.
- Création progressive des propriétés (object et data properties), axiomes, restrictions, en respectant ces relations.

6. **Génération de l'Ontologie en OWL :**

- Organisation des concepts et relations sous forme de fichiers OWL, type XML, exploitables avec des outils standards (ex. Protégé), avec une sortie prête à être manipulée ou enrichie par des experts.

Comparaison avec la littérature : Contrairement à d'autres méthodes principalement ciblées sur des domaines ou types de documents spécifiques, cette approche propose une solution universelle, applicable à divers domaines et types documentaires. Elle ne nécessite pas d'intervention experte lors de la génération initiale, produisant un ontologie standard OWL, facilement manipulable et modifiable, via un processus rapide et totalement automatisé.

Résultats expérimentaux : Trois expérimentations ont été menées : avec un court document, une collection de 15 textes et une collection de 24 documents XML. Ces expériences démontrent que la méthode réussit à représenter de manière synthétique les concepts essentiels du contenu documentaire, avec une organisation hiérarchique cohérente. La simplicité de la démarche n'altère pas la qualité des ontologies produites, qui illustrent bien le contenu sémantique.

Limitations : Le traitement de documents longs ou de vastes collections entraîne une augmentation considérable du nombre de termes dans les résumés et dans l'ensemble des termes d'index, ce qui complique la manipulation matricielle et la création de concepts. La performance peut donc se voir affectée par la dimension des données.

Perspectives d'amélioration : Pour perfectionner la qualité des relations sémantiques, il serait pertinent d'intégrer d'autres ontologies ou un thésaurus complémentaire à WordNet. Par ailleurs, le procédé pourrait bénéficier de l'intégration d'autres outils ou techniques permettant de définir des relations sémantiques supplémentaires, ou de renforcer la précision dans l'obtention des concepts.

1.4) Scalable techniques have been proposed to construct graphs from large text corpora (Debatty et al., 2014).

L'idée centrale de l'article est de résoudre le problème de la "malédiction de la dimensionnalité" et du volume lors de la comparaison de textes. Si vous avez 1 million de textes, une simple comparaison nécessiterait 10^{12} d'opérations. L'algorithme réduit cela via une approche en "entonnoir" :

1. La signature (le "résumé" intelligent)

Tout commence par le CTPH (*Context Triggered Piecewise Hashing*). Contrairement à un algorithme de hachage standard (comme le MD5) où changer une seule lettre change tout le résultat, le CTPH est "élastique". Si deux textes sont presque identiques, leurs signatures seront presque identiques.

- **Concept** : On transforme un texte de longueur variable en une signature de taille fixe qui capture la structure locale du texte.

2. Le partitionnement (l'élimination des comparaisons inutiles)

On utilise le principe du LSH (*Locality Sensitive Hashing*). L'algorithme découpe les signatures en morceaux (stages) et les utilise comme "clés".

- **Mécanisme :** Si deux documents partagent un morceau de signature identique, ils sont envoyés dans le même "seau" (bucket). S'ils ne partagent rien, on ne les comparera jamais. Cela permet d'ignorer 99% des documents qui n'ont aucune chance d'être similaires.

3. Le raffinement (la mesure de précision)

Une fois que des documents sont regroupés dans un seau, on sort les "instruments de précision". L'algorithme calcule la distance réelle (souvent la distance de Jaro-Winkler ou de Levenshtein) entre les textes de ce groupe.

- **Concept :** On vérifie si la proximité suggérée par le hachage est confirmée par une analyse textuelle rigoureuse.

4. L'organisation en graphe

Pour chaque document, l'algorithme ne retient que les k voisins les plus proches.

- **L'astuce :** Comme l'opération est distribuée sur plusieurs machines (MapReduce), chaque machine construit des petits morceaux de graphes qui sont ensuite fusionnés pour former la structure globale.

  Entrée et sortie :
  => L'INPUT (Entrée) : Données non structurées et paramètres

1. **Le corpus :** Une collection massive de documents texte bruts (ex: des millions de lignes de sujets d'emails, des logs de serveurs, ou des tweets). Ces données n'ont aucun lien entre elles au départ.
2. **Le paramètre k :** Le nombre de voisins souhaités pour chaque texte (ex: "k=10" signifie "trouver les 10 textes les plus proches pour chaque document").
3. **Une fonction de distance d(x, y)** (ex: Jaro-Winkler).

=> L'OUTPUT (Sortie) : Le graphe de similarité (K-NN Graph)

La sortie est une structure de données structurée composée de :

1. Les nœuds (les sommets)

Chaque document texte que vous avez mis en entrée devient un nœud unique dans le graphe.

- Imaginez chaque email ou chaque phrase comme un point sur une carte.
- Si vous aviez 1 million de textes en entrée, vous auriez 1 million de points dans votre graphe.

2. Les arêtes (les liens)

C'est ici que la structure apparaît. Pour chaque nœud (chaque texte), l'algorithme a cherché les k autres textes qui lui ressemblent le plus.

- Si vous avez choisi k=5, chaque point du graphe aura exactement 5 lignes (arêtes) partant de lui vers ses 5 "voisins" les plus proches.
- Ces liens ne sont pas créés au hasard : ils représentent une connexion réelle basée sur la similarité textuelle.

3. Les poids (la force du lien)

Le graphe de sortie est pondéré. Cela signifie que chaque lien (arête) porte une étiquette numérique.

- Cette étiquette est le score de similarité calculé (par exemple avec Jaro-Winkler).
- Exemple : Le document A est lié au document B avec un poids de 0.98 (ils sont presque identiques) et au document C avec un poids de 0.70 (ils partagent quelques thèmes ou phrases, mais sont différents).

À quoi ça sert une fois qu'on a ce graphe ?

Une fois que vous avez ce graphe en sortie, vous pouvez utiliser d'autres algorithmes pour :

1. Regrouper (Clustering) : Identifier automatiquement les groupes de spams ou les thématiques principales.
2. Visualiser : Voir graphiquement comment les informations circulent ou se regroupent dans votre base de données.
3. Filtrer : Si un nouveau texte arrive et qu'il se connecte très fortement à un groupe déjà identifié comme "Spam", vous pouvez le classer instantanément.

Le calcul du score de ressemblance dans l'algorithme NNCTPH repose sur deux piliers : une phase de filtrage rapide par hachage et une phase de calcul de distance précise entre les chaînes de caractères :

1. La métrique de similarité textuelle

Pour comparer deux textes et obtenir une valeur numérique (le score), l'article s'appuie sur des mesures de distance classiques pour les chaînes de caractères. Bien que l'algorithme soit conçu pour être générique, les auteurs utilisent principalement :

- La distance d'édition (Edit Distance / Levenshtein) : Elle calcule le nombre minimal d'opérations (ajout, suppression ou remplacement de caractères) nécessaires pour transformer un texte A en un texte B.
- La distance de Jaro-Winkler : Cette métrique est particulièrement adaptée aux textes courts (comme les sujets d'emails testés dans l'article). Elle donne un score entre 0 et 1, où 1 représente une identité parfaite. Elle accorde une importance accrue aux préfixes communs.

2. Le mécanisme de calcul (le processus)

Le score n'est pas calculé pour tous les documents, mais seulement pour ceux qui sont "suspectés" d'être proches.

1. Hachage CTPH (Fuzzy Hashing) : L'algorithme génère d'abord une signature pour chaque texte. Si deux signatures partagent des segments communs, ils sont placés dans le même "bucket" (réservoir).
2. Calcul exhaustif local : À l'intérieur de chaque réservoir, l'algorithme effectue une comparaison "brute force". Pour chaque paire (A, B) dans le réservoir, il applique la fonction de distance choisie (ex: Jaro-Winkler).
3. Normalisation : Le résultat de la distance est souvent converti en score de similarité :

   “Similarité = 1 - Distance Normalisée”

   Un score de 1 signifie que les textes sont identiques, tandis qu'un score proche de 0 signifie qu'ils n'ont rien en commun.

3. La sélection des voisins (k-NN)

Une fois les scores calculés, pour chaque document d(i), l'algorithme :

- Trie tous ses candidats par score décroissant.
- Ne conserve que les k documents ayant les scores les plus élevés (les plus proches voisins).
- Crée une arête dans le graphe avec ce score comme "poids" de la relation.

2) Etude comparative des méthodes :

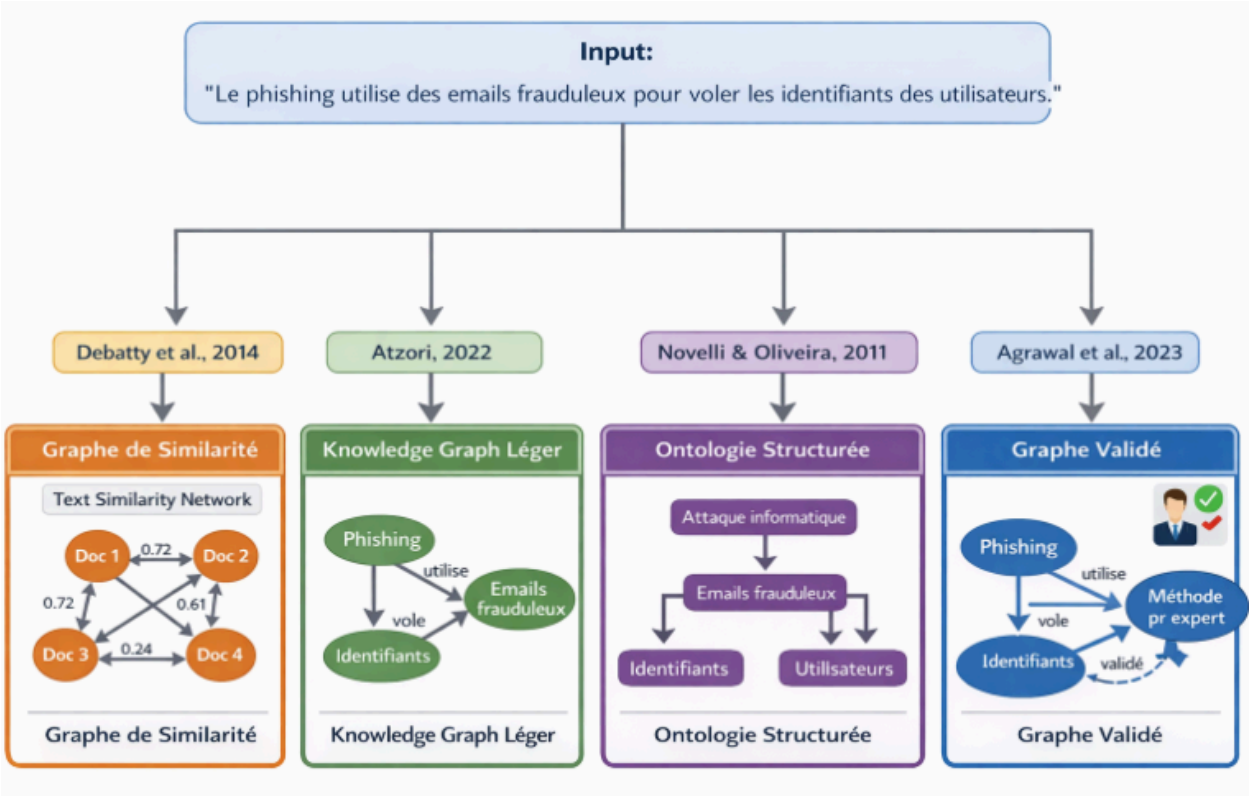
Critère d'analyse	Article 1	Article 2	Article 3	Article 4
Référence	**Debatty et al., 2014**	**Atzori, 2022**	**Novelli & Oliveira, 2011**	**Agrawal et al., 2023**
Domaine d'application	Analyse de données textuelles massives (Big Data), notamment pour la détection de spams	Extraction d'information, Knowledge Graphs, NLP, Question Answering	Extraction d'Information (générique)	Cybersécurité (enseignement)
Objectif principal	Construire efficacement et de manière scalable un graphe de k-plus proches voisins (k-NN graph) approximatif à partir de grands ensembles de données textuelles non structurées	Extraire automatiquement des données structurées à partir de texte libre non annoté et interrogation via des interfaces simples	Génération automatique d'ontologies simples à partir de corpus textuels	Construire un graphe de connaissances pédagogique à partir de textes non structurés
Type d'approche	Approche probabiliste basée sur le hachage (hashing) et le paradigme MapReduce pour le calcul distribué	Approche non supervisée basée sur le NLP et les embeddings de mots	Hybride (LSA, clustering, Wordnet, OWL)	Hybride (NLP automatique + expertise humaine)
Stratégie de construction	Bottom-up	bottom up	Bottom-up	Bottom-up
Niveau d'automatisation	Automatisé (processus algorithmique de construction de graphe)	Élevé: Extraction entièrement automatique sans données annotées ni modèles supervis	Entièrement automatisé (validation humaine optionnelle)	Semi-automatique
Type de données en entrée	Données textuelles non structurées (ex: sujets d'emails)	Texte libre non structuré	Documents texte brut ou structurés (XML)	Textes non structurés (cours, manuels de laboratoire, documents techniques)
Prétraitement du texte	Utilisation d'une fonction de hachage spécifique appelée *Context Triggered Piecewise Hashing* (CTPH) pour transformer les chaînes de caractères	Tokenisation et construction d'embeddings de mots	Lemmatisation, TF-IDF, sélection des termes fréquents	Segmentation, analyse syntaxique, POS tagging

Méthode d'extraction des entités	Non spécifié comme une extraction d'entités nommées classique ; chaque document (ou chaîne de texte) est considéré comme un nœud du graphe	Set Expansion	LSA (SVD) et similarité sémantique	Extraction basée sur POS tagging et règles linguistiques
Méthode d'extraction des relations	Calcul de similarité entre les chaînes de caractères (via des métriques de distance) pour identifier les voisins les plus proches	Relation Expansion	Relations lexicales Wordnet et mesures de similarité	Identification des relations via verbes et racines syntaxiques
Représentation des connaissances	Graphe pondéré où les nœuds sont des textes et les arêtes représentent la similarité	Knowledge Graph (nœuds = entités, arêtes = relations typées)	Ontologies OWL (concepts, propriétés, axiomes)	Triplets (entité – relation – entité)
Utilisation d'une ontologie	Non mentionnée	Oui	Oui	Oui
Type d'ontologie	Aucun. L'outil ne cherche pas à classer les mots dans des catégories logiques (hiérarchies, classes, propriétés). Il se contente de calculer si deux chaînes de caractères se ressemblent "visuellement" ou statistiquement via le hachage (CTPH)	Ontologie légère	Ontologie de domaine (généraliste ou spécifique)	Ontologie construite manuellement par des experts
Existence d'une ontologie préalable	Non. L'algorithme ne consulte pas de base de données comme DBpedia ou WordNet avant de travailler	Non	Oui (Wordnet)	Non
Gestion de l'ambiguïté sémantique	L'article se concentre sur la similarité syntaxique des chaînes de caractères (via CTPH) plutôt que sur la sémantique profonde	Partielle: Basée sur la proximité sémantique des embeddings	Wordnet et seuil de similarité	Règles ontologiques + validation par expert humain
Techniques d'apprentissage automatique	Algorithmes de recherche de plus proches voisins (k-NN) et hachage sensible à la localité (LSH)	Embeddings de mots non supervisés	Oui (Clustering non supervisé: Kmeans ou Agglomération Algorithm)	Oui (SVM pour la classification des intentions du chatbot)
Outils / technologies	MapReduce (Hadoop), algorithme NN-Descent (pour comparaison), CTPH	OKgraph (librairie Python) SPARQL pour l'interrogation	Wordnet, OWL, Java, Protégé	spaCy, Python, Neo4j, ReGraph

		Systèmes KGQA et BESt Queries		
Stockage du graphe	Non détaillé, mais traité dans un environnement distribué de type MapReduce	Knowledge Graph compatible avec des endpoints RDF SPARQL	Fichiers OWL	Base de données orientée graphe
Validation de la cohérence	Évaluation par comparaison avec un "ground truth" (vérité terrain) et mesure du rappel (recall) pour la qualité du graphe	Non explicitement traitée	Validation manuelle ou semi-automatique OWL	Validation du schéma via ReGraph
Couche de consommation	Non explicitement décrite (le graphe produit est destiné à être utilisé par d'autres algorithmes d'analyse)	Interfaces de Question Answering (KGQA) Interfaces By Example (BESt Queries via Infoboxes)	Systèmes d'information et moteurs sémantiques	Interface Web + chatbot
Utilisateurs cibles	Chercheurs et ingénieurs en données travaillant sur le minage de données massives et l'analyse de réseaux	Utilisateurs non techniques	Chercheurs et utilisateurs en recherche d'information	Étudiants en cybersécurité
Méthode d'évaluation	Tests expérimentaux sur des jeux de données réels (spams), mesure du temps d'exécution, de la scalabilité et de la qualité du graphe produit	Évaluation expérimentale sur benchmarks existants	Évaluation qualitative et validation experte	Évaluation qualitative basée sur des cas d'usage
Avantages principaux	Rapidité (4 fois plus rapide que NN-Descent pour une qualité égale) et haute scalabilité pour de très grands volumes de données	Extraction sans données annotées Indépendant de la langue Faible coût de mise en place Interfaces d'interrogation accessibles	Automatisation, simplicité, indépendance du domaine	Réduction de l'effort manuel, adaptation au domaine, graphe cohérent
Limites principales	La qualité dépend du choix des paramètres de hachage	Qualité dépendante des embeddings Ontologie peu formelle Gestion limitée de l'ambiguïté sémantique Pas de validation forte de cohérence du graphe	Scalabilité limitée et dépendance à Wordnet	Forte dépendance à l'expertise humaine, scalabilité limitée

Cette figure illustre la différence entre les quatre approches étudiées en appliquant le même texte d'entrée. À partir d'un même input, chaque méthode produit un type de sortie différent. La méthode proposée par Debatty et al. (2014) génère un graphe de similarité basé sur la proximité entre les documents. L'approche présentée par Atzori (2022), à travers la bibliothèque OKgraph, permet d'extraire automatiquement des

entités et des relations afin de construire un knowledge graph léger. La méthode décrite par Novelli et Oliveira (2011) produit une ontologie structurée en identifiant des concepts ainsi que des relations hiérarchiques entre eux. Enfin, l'approche d'Agrawal et al. (2023) combine l'extraction automatique d'informations à partir de textes non structurés avec une validation experte afin d'obtenir un graphe de connaissances plus précis et fiable.



Comparaison des méthodes de construction de graphes de connaissances

II. Descriptif détaillé de la solution proposée pour la génération d'un graphe RDF à partir d'un texte:

1. Objectif général et principes directeurs

L'objectif de cette solution est de concevoir et implémenter un Proof of Concept (POC) permettant la **génération semi-automatique d'un graphe de connaissances RDF à partir d'un texte en langage naturel**.
La solution s'inscrit dans le domaine du Web Sémantique, en combinant des approches :

- **symboliques** (NLP à base de règles, ontologies),
- **statistiques / neuronales** (LLM),
- et **humaines** (validation utilisateur).

L'approche retenue est celle d'une **boîte à outils modulaire**, où l'utilisateur intervient à différentes étapes du pipeline pour orienter, enrichir ou contraindre la génération du graphe. Cette philosophie permet :

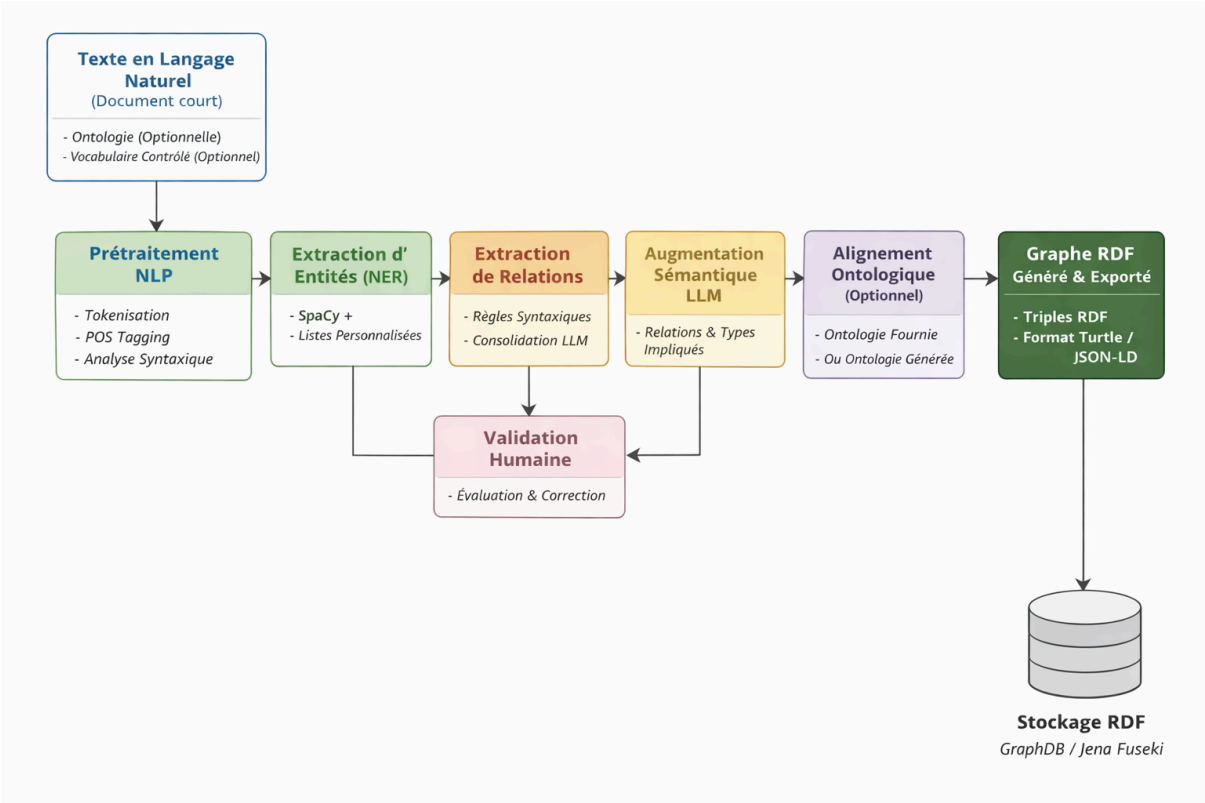
- d'améliorer la qualité sémantique des graphes produits,
- de rendre le système adaptable à différents domaines,
- et de respecter les principes de contrôle et d'explicabilité propres au Web Sémantique.

2. Vue d'ensemble du pipeline

Le pipeline global transforme un document textuel court en un graphe RDF structuré, stocké dans un triplestore. Il est composé des étapes suivantes :

1. **Entrée utilisateur et configuration**
2. **Prétraitement linguistique**
3. **Extraction des entités nommées**
4. **Extraction de relations (approche hybride)**
5. **Augmentation et consolidation sémantique via LLM**
6. **Alignement ontologique (optionnel)**
7. **Validation humaine**
8. **Génération, stockage et export du graphe RDF**

Chaque étape est **optionnelle ou paramétrable**, ce qui justifie l'appellation de *boîte à outils*.



3. Entrée utilisateur et configuration

3.1 Texte d'entrée

L'entrée principale est un **document textuel court**, par exemple :

- un paragraphe descriptif,
- un résumé d'article,
- un document institutionnel ou technique.

Le choix d'un texte court est volontaire afin de garantir la faisabilité du POC et de faciliter l'évaluation humaine.

3.2 Paramétrage utilisateur

Avant l'exécution du pipeline, l'utilisateur peut spécifier :

- la langue du texte,
- un vocabulaire contrôlé optionnel (liste d'entités d'intérêt),
- l'utilisation ou non d'une ontologie existante,

- le niveau d'automatisation (symbolique seul, hybride, enrichi par LLM),

Ce paramétrage constitue le premier niveau de personnalisation de la solution.

4. Prétraitement linguistique

4.1 Objectif

Le prétraitement vise à enrichir le texte avec des informations linguistiques nécessaires aux étapes suivantes, tout en conservant la structure syntaxique des phrases.

4.2 Méthodes utilisées

Les opérations suivantes sont réalisées :

- segmentation en phrases,
- tokenisation,
- lemmatisation,
- étiquetage morpho-syntaxique (POS tagging),
- analyse des dépendances syntaxiques.

4.3 Technologie

- **SpaCy** (en_core_web_lg ou fr_core_news_lg selon la langue)

Ces modèles offrent un bon compromis entre précision linguistique et performance, adapté à un POC.

5. Extraction des entités nommées (NER)

5.1 Rôle dans le graphe de connaissances

Les entités nommées constituent les **ressources (nœuds)** du graphe RDF. Leur qualité conditionne directement la pertinence du graphe final.

5.2 Approche retenue

a) NER standard

L'extraction repose sur les modèles pré-entraînés de SpaCy, capables d'identifier des entités telles que :

- PERSON, ORG, LOC,

- mais aussi PRODUCT, DATE, MONEY, LAW, etc.

L'intégration de types d'entités variés permet de capturer des connaissances plus riches, notamment dans des textes techniques ou institutionnels.

b) Personnalisation par vocabulaire contrôlé

En complément, l'utilisateur peut fournir une **liste d'entités spécifiques à son domaine** (ex : noms de projets, produits internes, concepts métiers).
Ces entités sont injectées dans le pipeline sous forme de règles ou de correspondances exactes, ce qui permet :

- d'améliorer la précision de l'extraction,
- de réduire les omissions sur des concepts importants mais peu fréquents.

5.3 Résultat intermédiaire

Chaque entité est représentée sous une forme structurée :

- un identifiant unique,
- un libellé textuel,
- un type d'entité (issu du NER),
- sa position dans le texte,
- un score de confiance associé à l'extraction.

6. Extraction des relations (composant central)

L'extraction des relations permet de relier les entités entre elles et constitue ainsi le cœur de la construction du graphe de connaissances. Cette étape repose sur une approche en deux niveaux : extraction syntaxique, puis consolidation sémantique.

6.1 Niveau 1 – Extraction syntaxique à base de règles

Principe

Deux entités nommées apparaissant dans une même phrase peuvent être reliées par une relation candidate si :

- elles sont connectées via un verbe,
- ou via une structure syntaxique explicite (préposition, apposition, complément).

Ces relations sont considérées comme **syntaxiques**, c'est-à-dire dépendantes de la formulation linguistique. Ces règles sont volontairement génériques, afin d'être applicables à différents domaines.

Exemples de patrons linguistiques

- Sujet – Verbe – Objet
  (ENT_A) —[verbe]→ (ENT_B)
- Structures avec prépositions
  (ENT_A) —[in / of / by]→ (ENT_B)
- Définitions
  "X is a Y"

Confiance syntaxique

Chaque relation extraite à ce niveau est associée à une confiance syntaxique *C_syntax*, calculée à partir de :

- la clarté de la structure syntaxique,
- le type de dépendance grammaticale,
- la distance entre les entités dans la phrase.

Technologie

- *SpaCy Dependency Parser*
- règles Python basées sur les dépendances (nsubj, dobj, prep, etc.)

6.2 Niveau 2 – Consolidation sémantique par LLM

Les relations extraites à l'étape précédente sont souvent lexicalement variables, peu normalisées, et difficilement exploitables comme propriétés RDF. Le rôle du LLM est donc de transformer une relation syntaxique en une relation **sémantique** stable, sans introduire de nouvelles connaissances.

Principe

Pour chaque triplet candidat (Entité A – verbe – Entité B) avec sa confiance syntaxique initiale *C_syntax* :

1. Le triplet et son contexte sont fournis au LLM via un prompt contraint
2. Le LLM retourne une relation sémantique normalisée
3. Une confiance LLM *C_llm* est calculée basée sur :
   - La cohérence de la réponse avec le contexte
   - La concision et la clarté de la formulation
   - L'absence de contenu halluciné

4. La confiance combinée *C_combined* est calculée comme :
   C_combined = α × C_syntax + β × C_llm (avec α+β=1, par défaut α=0.4, β=0.6)

Exemple

Texte : "Apple a lancé l'iPhone"
Relation extraite : Apple —**[a lancé, C_syntax=0.88]**→ iPhone
Relation consolidée : Apple —**[est le fabricant de, C_llm=0.96]**→ iPhone
Confiance combinée : C_combined = 0.4×0.88 + 0.6×0.96 = 0.92

Seuils de décision automatisée

- C_combined ≥ 0.85 : Relation automatiquement validée
- 0.70 ≤ C_combined < 0.85 : Nécessite validation humaine
- C_combined < 0.70 : Relation rejetée ou marquée comme hautement incertaine

Technologie

- API LLM (GPT-4 / GPT-3.5 / Llama 3 / Mistral)
- Prompts contrôlés et contraints

7. Augmentation sémantique par LLM

7.1 Objectif

L'augmentation sémantique vise à compléter les connaissances explicites du texte, en explorant :

- des **relations implicites**,
- des **types conceptuels (rdf:type)** pour les entités,

7.2 Augmentation des relations

Principe :

À partir d'une relation déjà validée ou partiellement validée entre deux entités, le LLM est interrogé pour proposer des relations supplémentaires plausibles, mais non explicitement mentionnées dans le texte.

Exemple :
Relation existante :
Apple — est le fabricant de → iPhone

Relations augmentées possibles :

- Apple — **développe** → iPhone
- Apple — **commercialise** → iPhone

Marquage explicite :

Toute relation augmentée est marquée comme hypothétique :

- is_hypothetical = true
- confiance pénalisée par rapport à la relation de base
- validation humaine requise

7.3 Augmentation des entités : inférence de types conceptuels

Principe :

Cette augmentation ne concerne pas les relations, mais la nature des entités elles-mêmes. Le LLM est utilisé pour proposer un type conceptuel général (rdf:type) à une entité, lorsque ce type n'est pas explicitement mentionné dans le texte.

Exemple

Texte : *"Apple a lancé l'iPhone."*

- Entité : Apple (ORG – NER)
- Type inféré : Organization
- Entité : iPhone (PRODUCT – NER)
- Type inféré : Product

Représentation RDF :

ex:Apple rdf:type ex:Organization .
ex:iPhone rdf:type ex:Product .

Marquage des types inférés

Les types inférés sont systématiquement annotés :

- is_inferred = true
- confidence_level = low / medium
- requires_human_validation = true

Ces annotations permettent de distinguer :

- ce qui est extrait directement du texte,
- de ce qui est inféré par augmentation sémantique.

8. Alignement ontologique

L'alignement ontologique constitue le cœur Web sémantique de la solution.

8.1 Mode Découverte (sans ontologie fournie)

- Les relations consolidées deviennent des rdf:Property.
- Les types générés deviennent des owl:Class.
- Une ontologie légère *ad-hoc* est créée automatiquement.

8.2 Mode Alignement (ontologie fournie)

Si l'utilisateur fournit une ontologie (.owl ou .ttl) :

- Les entités sont alignées avec des classes existantes
- Les relations sont mappées vers des propriétés ontologiques
- Les éléments non alignables sont filtrés

L'alignement repose sur :

- similarité lexicale (labels, commentaires),
- éventuellement aide du LLM pour choisir la propriété la plus pertinente.

Technologies

- RDFLib
- OWLReady2
- SPARQL

9. Validation humaine (Human-in-the-loop)

Une étape de validation humaine est intégrée afin de :

- confirmer ou rejeter certains éléments (les relations augmentées, les types conceptuels inferes…),
- corriger des erreurs,

- améliorer la qualité finale du graphe.

Au niveau de l'utilisateur, une visualisation simple et interactive est à mettre en place, avec des actions basiques :

- valider / invalider un lien,
- modifier un libellé.

10. Génération, stockage et export du graphe

10.1 Génération RDF

Le graphe final est généré en RDF, sous forme de triplets :

- sujet – prédicat – objet.

10.2 Stockage

- triplestore en mémoire
- Fichier XML
- GraphDB Free / Apache Jena Fuseki.

10.3 Formats d'export

- **XML ou Turtle (.ttl)** pour la lisibilité,
- **JSON-LD** pour une intégration web éventuelle.

11. Implémentation

L'implémentation du prototype a été réalisée sous la forme d'une interface interactive permettant à l'utilisateur de tester facilement la génération d'un graphe de connaissances à partir d'un texte. L'interface comporte une barre latérale (sidebar) dans laquelle l'utilisateur peut sélectionner le mode d'extraction : symbolique, hybride ou entièrement automatique avec l'utilisation de LLM.

Dans les trois cas, l'utilisateur peut également fournir un vocabulaire contrôlé afin de guider l'extraction des entités et améliorer la précision des résultats. La zone principale de l'interface permet d'introduire le texte d'entrée, puis de lancer le traitement du pipeline.

Le système applique ensuite les différentes étapes du traitement : prétraitement linguistique, extraction des entités nommées et identification des relations à l'aide de la bibliothèque spaCy. Les résultats sont

présentés sous forme d'entités et de relations extraites, permettant finalement la génération et l'exportation d'un graphe de connaissances sous forme d'ontologie.

Des captures d'écran de l'interface sont présentées afin d'illustrer les différentes fonctionnalités du prototype et le fonctionnement du système

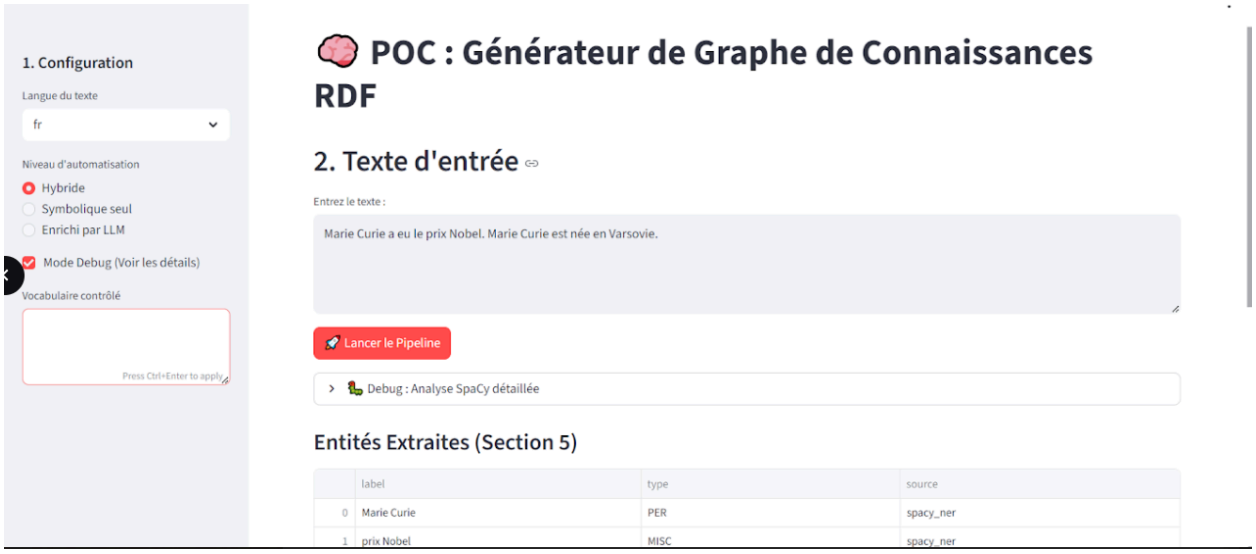
1. Architecture globale & stack technologique

L'application est conçue selon une architecture modulaire de type **Pipeline NLP**, où chaque brique est indépendante et interchangeable.

Couche	**Composant**	**Version**	**Rôle stratégique**
Interface UI	Streamlit	Latest	Orchestration visuelle. Gère l'état de session (`st.session_state`) pour la persistence des données lors des interactions humaines.
NLP Core	SpaCy	fr_core_news_lg	Analyseur linguistique profond. Fournit l'arbre de dépendances, le POS tagging et la lemmatisation. C'est la "vue syntaxique" de la machine.
Moteur IA	Ollama	llama3 / phi3	Cerveau neuronal. Utilisé pour la Consolidation Sémantique (normalisation des relations) et l'enrichissement. Fonctionne en local pour la confidentialité.
Données	RDFLib	6.x	Usine à graphes. Transforme les paires (Entités, Relations) en triplets RDF standards (Sujet-Prédicat-Objet).
Runtime	Python	3.10+	Langage hôte. Gère la logique métier, les calculs de confiance et l'intégration des API.

2. Le tableau de debug :

Le mode Debug affiche la structure profonde de la phrase telle que SpaCy la comprend. C'est indispensable pour diagnostiquer pourquoi une relation est détectée ou non.



Colonne	Définition technique	Exemple	Impact sur la détection
Texte	Le token (mot ou ponctuation) brut.	Apple	C'est l'unité de base.
LEMME	Forme canonique du mot (racine).	Est => être	Permet de regrouper "est", "sont", "être" sous un même concept relationnel.
POS	Part-Of-Speech (Nature grammaticale).	PROPN (Nom Propre), VERB	Le filtre principal. Le système cherche prioritairement des liens entre PROPN via des VERB.
Dépendance	Relation grammaticale vers le parent.	nsubj (Sujet), obj (Objet)	Indique si le mot est l'acteur (nsubj) ou la cible (obj) de l'action.
Tête	Le mot "chef" de ce token.	Pour Apple, la tête est “a lancé”.	Permet de remonter les fils. Si deux entités pointent vers le même verbe (la tête commune), elles sont probablement liées.

Analyse d'un cas : Phrase : "Tim Cook dirige Apple."

1. **Tim Cook** (PROPN) –nsubj→ **dirige** (VERB) ←obj– **Apple** (PROPN).
2. L'algorithme voit que "Tim Cook" et "Apple" sont connectés par la tête commune "dirige". Il extrait la relation

3. L'algorithme moteur : "Shortest Path" (Chemin le plus court)

Le concept

Au lieu de chercher "Qui a fait quoi ?", le système regarde **l'arbre syntaxique comme une carte géographique**. Il cherche le chemin le plus court pour aller de l'Entité A à l'Entité B.

La méthode

1. **Mapping** : Le système identifie les tokens qui appartiennent à des entités (ex: Token 1 à 2 = "Apple").
2. **Ancêtre commun** : Pour chaque paire d'entités, il cherche leur plus proche ancêtre commun dans l'arbre (souvent le verbe principal).
3. **Extraction du Chemin** : Il liste tous les mots significatifs entre les deux entités.
    - *Exemple :* "Apple, basée à Cupertino, a lancé l'iPhone."
    - Chemin trouvé : "Apple"→"lancé"→"iPhone".
    - Relation brute générée : **Apple --[lancé]--> iPhone**.

Cela permet de capturer des relations même si la phrase est complexe ou contient des incises.

4. Les formules de calcul (Scoring)

Le système attribue un score de confiance (entre 0 et 1) à chaque relation.

A. Score Syntaxique (*Csyntax*) - *Approche Proximité*

Formule implémentée :

$$Csyntax = \max(0.4, 1.0 - (D \times 0.02))$$

- ***D* (Distance)** : Nombre de mots entre l'Entité A et l'Entité B.
- **0.02 (Pénalité)** : Chaque mot d'écart réduit la confiance de 2%.
- **0.4 (Plancher)** : Le score ne descend jamais sous 0.4. On garde une chance pour le LLM de valider des relations lointaines.

Exemple : Si "Apple" est à la position 0 et "iPhone" à la position 3.

$D=3$

$C_{syntax}=1.0-(3\times0.02)=0.94$ (Confiance excellente).

B. Score Hybride Final (*Ccombined*) - *Approche Neuronale*

Une fois le LLM consulté pour nettoyer la relation (ex: changer "a lancé" en "commercialise"), on combine les scores.

Formule :

$$C_{combined}=\alpha\times C_{syntax}+\beta\times C_{llm}$$

- ***Alpha* (Poids syntaxique) = 0.4**
- ***Beta* (Poids LLM) = 0.6**
- ***Cllm*** : Score de confiance du LLM (fixé à 0.9 par défaut dans le POC si la réponse est cohérente).

Calcul final :

$$C_{combined}=(0.4\times0.94)+(0.6\times0.9)$$

$$C_{combined}=0.376+0.54=0.916$$

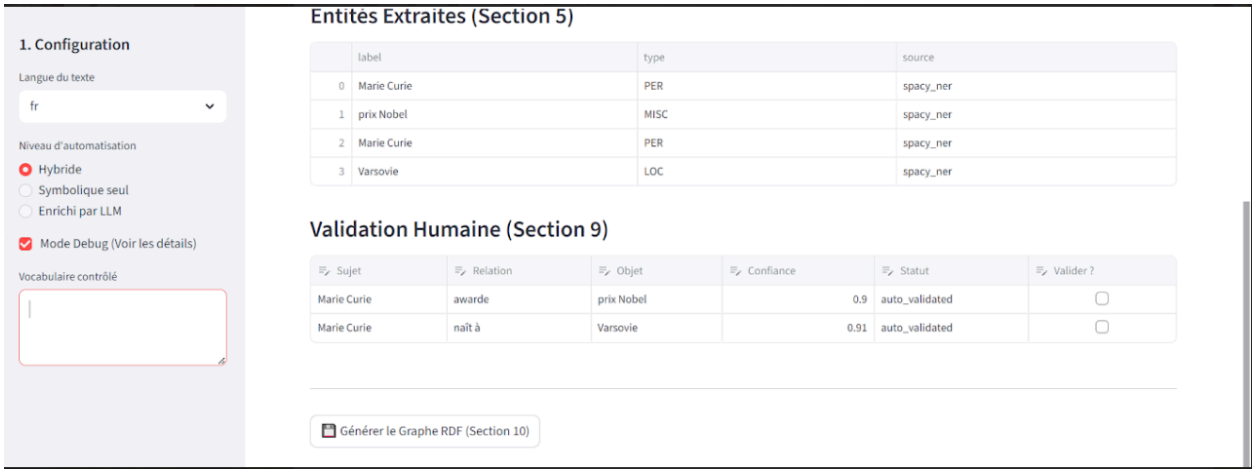
C. Seuils de décision

Le statut de la relation est déterminé par ce score final :

- $C_{combined}\geq0.85$ **→ auto_validated** (Vert, validé sans intervention).
- $0.70\leq C_{combined}<0.85$ **→ needs_review** (Jaune, demande validation humaine).
- $C_{combined}<0.70$ **→ rejected** (Rouge, rejeté).

5. Flux de données technique (Pipeline)

1. **Prétraitement (nlp_processor.py)** :
    - Le texte est tokenizé.
    - **Fallback Noun Chunks** : Si SpaCy ne trouve pas d'entités nommées (NER), le système extrait les groupes nominaux importants (ex: "le président", "la voiture") pour servir d'entités.
2. **Extraction (relation_extractor.py)** :
    - Construction de l'arbre.
    - Algorithme Shortest Path entre les entités détectées.
    - Génération d'une liste de relations candidates brutes.



3. **Consolidation (llm_service.py)** :
   - Appel à l'API Ollama locale.
   - Envoi d'un prompt contraint : *"Normalise la relation 'Apple' 'lancé' 'iPhone' en propriété RDF concise."*
   - Réception de la réponse : *"commercialise"*.
4. **Validation (app.py)** :
   - Calcul des scores.
   - Présentation à l'utilisateur dans le **st.data_editor**.
5. **Sérialisation (rdf_builder.py)** :
   - Conversion des objets Python en URI RDF.
   - Génération du code Turtle.

Limites et perspectives

La solution proposée présente certaines limites. Par exemple, lorsque l'extraction des relations repose uniquement sur des règles syntaxiques (approche symbolique), le système n'est pas toujours capable d'identifier correctement les relations entre les entités. Certaines relations peuvent être mal interprétées ou ne peuvent pas être détectées lorsque la structure de la phrase est complexe.

Par ailleurs, l'utilisation des LLM peut améliorer l'interprétation sémantique, mais elle peut aussi introduire des erreurs ou des relations qui ne sont pas explicitement présentes dans le texte.

Comme perspectives, plusieurs améliorations peuvent être envisagées, notamment l'amélioration des règles d'extraction, l'entraînement de modèles NER spécialisés pour le domaine étudié, ou encore l'intégration de mécanismes de validation plus avancés afin d'augmenter la qualité du graphe de connaissances généré.

Bibliographie:

- Agrawal, G., Deng, Y., Park, J., Liu, H., & Chen, Y.-C. (2023).

  Building Knowledge Graphs from Unstructured Texts: Applications and Impact Analyses in Cybersecurity Education.

- Atzori, M. (2022).

  Extracting Data from Text and Querying it via KGQA and BESt Queries. University of Cagliari.

- Novelli, A. D. P., & Oliveira, J. M. P. (2011).

  Simple Method for Ontology Automatic Extraction from Documents.

- Debatty, T., Michiardi, P., Thonnard, O., & Mees, W. (2014).

  Scalable Graph Building from Text Data. Proceedings of the 3rd International Workshop on Big Data, Streams and Heterogeneous Source Mining (BigMine). *Proceedings of Machine Learning Research*, 36, 120–132.